Customer Churn Analysis: The Complete Analytical Journey

This document traces the full analytical path behind the churn analysis from raw data through Power BI modelling, DAX measures and predictive scoring, targeted SQL validation, financial quantification, and experimental design. It exists to show not just *what* was found, but *why* each tool was used where it was, and how each stage built on the one before it.

The project deliberately used two tools for different jobs rather than forcing everything through one: **Power BI** for exploration, relational modelling, DAX-driven risk scoring, and dashboard communication; **SQL** for direct, reproducible answers to specific business questions once Power BI's exploration had identified which questions mattered.

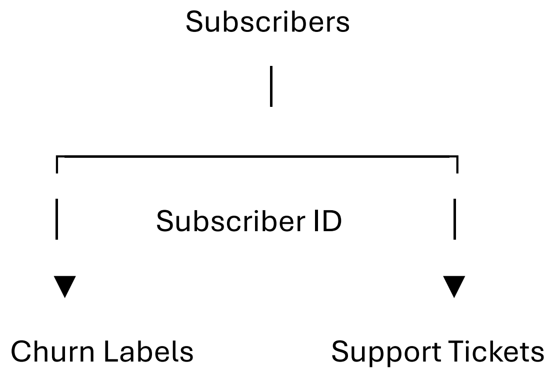
1. Data Preparation and the Power BI Model

Before any exploration began, the three source datasets: Subscribers, Churn Labels and Support Tickets were prepared in Power Query.

Transformations applied:

- Standardized data types for consistency
- Converted binary fields (1/0) into readable Yes/No labels
- Created grouped/bucketed variables for continuous fields (tenure, monthly charge) to reduce unnecessary granularity
- Created helper sort columns so custom buckets displayed in natural business order (Low → Mid → High) rather than alphabetically
- Removed unused columns and disabled loading of intermediate queries to reduce model size
- Established relationships across tables using Subscriber ID

The resulting model:



This relational model became the analytical hub of the project the environment used for exploration, DAX measures, the predictive risk score, and the A/B test simulation.

SQL views were deliberately kept outside this model. Each view had already answered one specific business question before being imported; connecting them into the relational structure would have added complexity without adding analytical value. Instead, they functioned as independent, pre-answered outputs feeding specific dashboard visuals directly.

The architecture therefore had two complementary components: the **relational Power BI model** (exploration, modelling, DAX, prediction, experimentation) and **independent SQL views** (direct, targeted answers to specific questions surfaced during that exploration).

2. DAX: Measures, Debugging, and the Predictive Layer

Once the model was built, DAX was used for three distinct jobs: calculating core churn metrics correctly across a filtered model, translating findings into the predictive risk score, and building the A/B test simulation. Each is a different kind of DAX work, and each surfaced its own problem to solve.

2.1 Core Measures

DAX

```
Total Subscribers = COUNT(Subscribers[Subscriber ID])


Churned Customers = CALCULATE(
   COUNTROWS('Churn Labels'),
   'Churn Labels'[Churned] = "Yes",
   CROSSFILTER('Churn Labels'[Subscriber ID], Subscribers[Subscriber ID], BOTH)
)


Churn Rate = DIVIDE([Churned Customers], [Total Subscribers], 0)


Churn Rate (Fixed) = CALCULATE(
   DIVIDE([Churned Customers], [Total Subscribers], 0),
   ALL(Subscribers)
```

)

2.2 The Bug That Shaped the Model

Early in the build, Churned Customers returned the same number regardless of which filter or slicer was applied clicking into any segment left the churned count unchanged, which meant every derived churn rate was silently wrong.

The cause: Churn Labels sits on the "one" side of a one-to-many relationship with Subscribers. By default, Power BI relationships only propagate filters in one direction from the "one" side down to the "many" side so a filter applied on Subscribers (e.g. clicking a Plan Type) had no way to flow *back* to Churn Labels and affect the churned count.

The fix used CROSSFILTER(...BOTH), scoped only to this one measure, rather than changing the relationship's global cross-filter direction in the model a global change would have re-introduced an ambiguous filter path elsewhere in the model (Churn Labels also connects to Support Tickets), so the fix was deliberately narrow: solve it exactly where it broke, without destabilizing anything else.

Debugging principle that came out of this: when a number stays identical no matter what filter is applied, check whether the filtering column and the calculated column sit in different tables with a one-directional relationship between them that mismatch is often the actual cause, not the formula itself.

Churn Rate (Fixed) uses ALL(Subscribers) for a different purpose a version of the rate that is deliberately *immune* to any filter on the Subscribers table, used where a constant, unfiltered baseline needs to appear next to a filtered number (e.g. comparing a segment's rate against the true 22.57% baseline inside the same visual).

2.3 Risk Flags and Score

Each risk flag is a calculated column, evaluated row-by-row against the Subscribers table, some of them reaching into related tables via CALCULATE + COUNTROWS:

DAX

```
Flag_NoContractNewTenure = IF(
  Subscribers[Contract Months] = 0 && Subscribers[Tenure Groups] = "New", 1, 0
)


Flag_PriceMismatch = IF(
  (Subscribers[Plan Type] = "Prepaid" && Subscribers[Monthly Charge Group] = "Low")
```

```
    || (Subscribers[Plan Type] = "Postpaid Premium"), 1, 0
)

Flag_BillingDispute = IF(
    CALCULATE(COUNTROWS('Support Tickets'), 'Support Tickets'[Issue Type] = "Billing Dispute") > 0,
    1, 0
)

Flag_Escalated = IF(
    CALCULATE(COUNTROWS('Support Tickets'), 'Support Tickets'[Escalated] = "Yes") > 0,
    1, 0
)

Flag_LowSatisfaction = IF(
    CALCULATE(COUNTROWS('Support Tickets'), 'Support Tickets'[Satisfaction Rating] <= 2) > 0,
    1, 0
)

RiskScore =
    Subscribers[Flag_NoContractNewTenure] +
    Subscribers[Flag_PriceMismatch] +
    Subscribers[Flag_BillingDispute] +
    Subscribers[Flag_Escalated] +
    Subscribers[Flag_LowSatisfaction]

RiskTier = IF([RiskScore] >= 4, "High", IF([RiskScore] >= 2, "Medium", "Low"))
```

Documentation note carried through the model: a subscriber is flagged if **any** related ticket meets the condition one billing dispute is sufficient regardless of when it occurred, or how many other tickets exist.

An earlier version of this model included a sixth flag, for new-tenure subscribers acquired via Online or Retail channels. It was removed after SQL validation showed the acquisition-channel signal rode almost entirely on tenure rather than adding independent risk the flag was diluting the score with a variable that wasn't pulling its own weight. RiskScore and RiskTier thresholds were adjusted accordingly once the flag was dropped.

2.4 A/B Test Simulation

DAX

```
RandomValue = RAND()


TestGroup = IF(
   Subscribers[RiskTier] = "High",
   IF(Subscribers[RandomValue] < 0.5, "Control", "Treatment"),
   "N/A"
)
```

RandomValue is a calculated column, so it only reshuffles on data refresh not on every interaction which keeps a subscriber's Control/Treatment assignment stable while exploring the report. Documented explicitly: this assignment is frozen for analysis purposes, and a live refresh would reshuffle it.

3. Why SQL Was Brought In

Power BI's exploration and DAX measures surfaced real, meaningful patterns but many of them still required a reader to mentally compare values after interacting with a slicer, or infer a segment's churn rate from raw counts rather than seeing it stated directly.

Rather than expecting a stakeholder to do that math themselves, targeted SQL queries were written to answer each identified business question directly calculating the churn rate *within* a segment, not a segment's share of total churn, so the number on screen is the number a reader can act on immediately.

This was a conscious trade-off: SQL views are static, not interactive like the rest of the dashboard. But because each one existed to answer one already identified question directly, the loss of interactivity was an acceptable cost for the gain in clarity.

The full workflow:

Raw CSV Data

↓

Power Query Cleaning

↓

Power BI Relationships

↓

Power BI Exploration

↓

Business Question Identified

↓

Targeted SQL View Created

↓

SQL Result Validated & Imported

↓

Business Interpretation

↓

Predictive Risk Model

↓

Business Recommendations

↓

A/B Test Simulation

↓

Iteration

Data preparation note: source CSVs imported into SQL Server with numeric fields as nvarchar an import characteristic, not a data-quality issue. Every numeric comparison in SQL explicitly casts (CAST(tenure_months AS INT), CAST(monthly_charge_usd AS FLOAT)) as a result.

4. SQL Views Used to Answer Business Questions

Six independent views, each created to answer one specific question surfaced during Power BI exploration. They are not relationally connected to one another or to the Power BI model.

4.1 Contract × Plan Type

Question: How does churn vary across plan types and contract commitment periods?

sql

```
CREATE VIEW [dbo].[ContractXPlanType] AS
SELECT s.plan_type, s.contract_months, COUNT(*) AS total_subs,
  SUM(CASE WHEN churned = 1 THEN 1 ELSE 0 END) AS churned_subs,
  ROUND(100.0 * SUM(CASE WHEN churned = 1 THEN 1 ELSE 0 END) / COUNT(*), 2) AS churn_rate
FROM Subscribers s
INNER JOIN churn_labels c ON s.subscriber_id = c.subscriber_id
GROUP BY s.plan_type, s.contract_months;
```

Supported: Recommendation 1 (Onboarding) and Recommendation 5 (Contract Commitment).

4.2 Reason for Churn

Question: What reasons are most frequently reported by subscribers who churn?

sql

```
CREATE VIEW [dbo].[ReasonForChurn] AS
SELECT churn_reason, COUNT(*) AS total,
  ROUND(100.0 * COUNT(*) / (SELECT COUNT(*) FROM churn_labels WHERE churned = 1), 2) AS pct_of_churned
FROM churn_labels
WHERE churned = 1
GROUP BY churn_reason;
```

Supported: Recommendation 3 (Pricing) confirmed competitor offers and price as the dominant stated reasons.

4.3 Customer Rating

Question: How does customer satisfaction relate to churn and retention?

sql

```
CREATE VIEW [dbo].[CustomerRating] AS

SELECT s.customer_satisfaction, COUNT(*) AS total_subs,

  SUM(CASE WHEN churned = 1 THEN 1 ELSE 0 END) AS churned_subs,

  SUM(CASE WHEN churned = 0 THEN 1 ELSE 0 END) AS retained,

  ROUND(100.0 * SUM(CASE WHEN churned = 1 THEN 1 ELSE 0 END) / COUNT(*), 2) AS churned_rate,

  ROUND(100.0 * SUM(CASE WHEN churned = 0 THEN 1 ELSE 0 END) / COUNT(*), 2) AS retention_rate

FROM churn_labels c

LEFT JOIN support_tickets s ON c.subscriber_id = s.subscriber_id

GROUP BY s.customer_satisfaction;
```

Supported: Recommendation 2 (Support Experience) treated as association, not causation.

4.4 Escalated Tickets

Question: Do subscribers with escalated support tickets churn at a higher rate?

sql

```
CREATE VIEW [dbo].[EscalatedTickets] AS

WITH SubscriberEscalation AS (

  SELECT s.subscriber_id, c.churned,

    CASE WHEN EXISTS (SELECT 1 FROM support_tickets t

      WHERE t.subscriber_id = s.subscriber_id AND t.escalated = '1')

    THEN 'Escalated' ELSE 'Not Escalated' END AS escalation_status

  FROM Subscribers s

  JOIN churn_labels c ON s.subscriber_id = c.subscriber_id

)

SELECT escalation_status, COUNT(*) AS total_subs,

  SUM(CASE WHEN churned = 1 THEN 1 ELSE 0 END) AS churned,
```

ROUND(100.0 * SUM(CASE WHEN churned = 1 THEN 1 ELSE 0 END) / COUNT(*), 2) AS churn_rate_pct

FROM SubscriberEscalation

GROUP BY escalation_status;

Result: 43% vs. 21% churn gap. Supported Recommendation 2.

4.5 Plan Type × Charge Group

Question: Does churn vary by plan type and monthly price level?

sql

CREATE VIEW [dbo].[PlanTypeChargeGroup] AS

SELECT plan_type, COUNT(*) AS total_subs,

CASE WHEN CAST(monthly_charge_usd AS FLOAT) <= 35 THEN 'Low'

WHEN CAST(monthly_charge_usd AS FLOAT) <= 65 THEN 'Mid' ELSE 'High' END AS charge_group,

SUM(CASE WHEN churned = 1 THEN 1 ELSE 0 END) AS churned_subs,

ROUND(100.0 * SUM(CASE WHEN churned = 1 THEN 1 ELSE 0 END) / COUNT(*), 2) AS churn_rate

FROM Subscribers s

INNER JOIN churn_labels c ON s.subscriber_id = c.subscriber_id

GROUP BY plan_type, CASE WHEN CAST(monthly_charge_usd AS FLOAT) <= 35 THEN 'Low'

WHEN CAST(monthly_charge_usd AS FLOAT) <= 65 THEN 'Mid' ELSE 'High' END;

Supported: Recommendation 3. Small-sample segments (n=2) excluded from financial estimates.

4.6 Tenure × Channel

Question: Does churn vary meaningfully across acquisition channels and lifecycle stages?

sql

CREATE VIEW [dbo].[TenureXChannel] AS

SELECT COUNT(*) AS total_subs,

SUM(CASE WHEN c.churned = 1 THEN 1 ELSE 0 END) AS churned,

```
  ROUND(100.0 * SUM(CASE WHEN c.churned = 1 THEN 1 ELSE 0 END) / COUNT(*), 2) AS churn_rate,
  acquisition_channel,
  CASE WHEN CAST(tenure_months AS INT) <= 12 THEN 'New'
       WHEN CAST(tenure_months AS INT) <= 36 THEN 'Established' ELSE 'Loyal' END AS tenure_group
FROM Subscribers s
INNER JOIN churn_labels c ON s.subscriber_id = c.subscriber_id
GROUP BY acquisition_channel, CASE WHEN CAST(tenure_months AS INT) <= 12 THEN 'New'
       WHEN CAST(tenure_months AS INT) <= 36 THEN 'Established' ELSE 'Loyal' END;
```

Result: Ruled out. Average monthly charge was essentially identical across channels (~$58.25–$58.55); the finding wasn't strong enough to justify a standalone recommendation. Kept in the repo deliberately it documents an investigated and rejected path, showing the analysis wasn't built to confirm a predetermined conclusion.

5. Financial Impact Estimation

Methodology: Churn gap to target × addressable subscriber population × average monthly charge = estimated monthly revenue at stake.

Recommendation	**Segment / Target**	**Est. Monthly Revenue at Stake**
1.Onboarding	No-Contract + New → Established rate	~$57,100/month *(near-term only; stretch scenario to Loyal rate ≈ $348,500/month is illustrative, not summed below)*
2.Support Experience	1-star & 2-star → 3-star churn rate	~$749,300/month
3.Pricing	Postpaid Premium High + Prepaid Low → baseline	~$402,200/month
5.Contract Commitment	Postpaid Premium, no commitment → 12-month rate	~$134,750/month (~$1.62M/year)

Total (near-term figures only)		**≈ $1.34M/month**

Acquisition channel produced no separate estimate average subscriber value was nearly identical across channels, so a financial figure wasn't manufactured for a finding that didn't demonstrate a meaningful opportunity.

These are directional opportunity estimates, not guaranteed recovery. They assume the observed churn-rate gaps are actionable, selected benchmarks are achievable, and retained subscribers continue generating their current average charge. Segments may overlap, so figures represent independent opportunity sizes rather than an additive guarantee.

6. From Findings to the Predictive Risk Model

The descriptive analysis showed *where* churn concentrated. The risk model exists to answer the next question: *given what we now know, who should the business prioritize right now?*

The strongest, independently confirmed patterns were converted into five binary risk flags (Section 2.3) derived directly from the findings above, not selected independently of them.

The score reflects how many independently identified risk characteristics a subscriber matches not a claim that each additional flag *causes* churn, but a prioritization signal: a subscriber matching several confirmed risk patterns warrants more attention than one matching only one.

This creates the analytical progression: **individual indicators → risk score → risk tier → prioritized subscriber population** turning a population-level question ("which characteristics associate with churn?") into an operational one ("which individual subscribers currently show multiple risk characteristics right now?").

Model caveat: this is a prioritization framework derived from current patterns, not a permanently valid predictive model. If interventions succeed say, onboarding improvements reduce churn among new no-contract subscribers the relationship that originally made that segment high-risk will weaken, and the model will need recalibration against new data. The logic is reusable; the specific weights are not assumed permanent.

7. From Risk Model to A/B Test Simulation

Once high-risk subscribers were identified, the next question became: *if the business acts on this list, how would we know whether it actually worked?*

Subscribers classified High Risk were randomly split into Control and Treatment (Section 2.4). In a real deployment: **Control** continues receiving the existing experience;

Treatment receives a specific intervention (improved onboarding, support resolution, pricing adjustment, or contract incentive). Churn would then be measured over a defined observation period and compared for statistical significance before any intervention is scaled.

Limitation stated plainly: the dataset is static and no real intervention was deployed the simulated groups do not constitute an actual experiment, and the comparison is not evidence that any intervention caused a change in churn. Its purpose is to demonstrate the experimental design that would be used to move from observation to causal validation.

8. How Every Component Connects

Stage	Question it answers	Output
Descriptive & diagnostic analysis	Where is churn happening, and what's associated with it?	Churn findings by segment
Financial quantification	How significant is this, in dollars?	Estimated revenue at stake
Predictive risk model	Which individual subscribers show multiple risk characteristics right now?	Risk flags → score → tier → prioritized population
Business recommendation	What should the business do about it?	Targeted interventions
A/B test	Does the intervention actually work?	Experimental design for causal evidence
Iteration	Did behavior change, and does the original logic still hold?	Recalibrated model, refined recommendations

9. The Complete Journey, End to End

Raw subscriber, churn, and support data

↓

Power Query cleaning & Power BI data model

↓

DAX measures, risk flags, and risk score

↓

Power BI exploratory analysis

↓

Identify patterns and relationships

↓

Formulate specific business questions

↓

Targeted SQL analysis and validation

↓

Determine which findings are meaningful; rule out weak ones

↓

Quantify financial opportunity

↓

Classify subscribers into risk tiers; prioritize high-risk subscribers

↓

Develop targeted business recommendations

↓

Simulate Control vs. Treatment assignment

↓

Demonstrate how recommendations could be experimentally tested

↓

(Real deployment) Measure outcomes → recalibrate → iterate